

Assignment - 12

Q. What is meant by reference in C++? with examples

- - Newly added datatype in C++
- Applicable in C++ & Java.
- Derived datatype
- When we create a reference to an existing variable it is considered as another name to that variable
- Just another name so that there is no separate memory allocation for reference

1) Creation of Reference → '&' (Reference Operator)

2) If the & operator used!

a) after assignment operator (=R+15) → Address of operator.

b) before assignment operator (lhs =) → Reference operator.
refer * reference.cpp

★ Name of reference as well as name of original variable → 'Refers to same memory location'.
∴ address & value of both is same.

2] What is the difference between pointer & reference.

	Pointer	Reference
Def ⁿ	A variable that stores the address of another variable	An alias for another variable.
Syntax	uses * to declare (E.g. int *ptr;)	uses & to declare (E.g. int &ref;)
Nullability	Can be null (E.g. int *ptr = NULL;)	Cannot be NULL, must be initialized when declared
Reassignment	Can point to different variables.	Cannot be reassigned after initialization
memory access	Requires dereferencing (e.g. *p).	Direct access, no dereferencing required

3] Explain different ways to call a function in C++.

1) Call by Value:

A copy of actual parameter is passed to the function.

E.g. void func(int x)

{ x = x + 10; // modifies the local copy }

2) Call by Reference (using References):

→ the actual parameter is passed, allowing the function to modify the original variable.

E.g. void func(int &x)

{ x = x + 10; // modifies the original variable }

3] Call by Pointer :

Address of variable is passed & function modifies the original value using dereferencing.

void func(int *x)

{ *x = *x + 10; // modifies original variable }

4] Draw symbol table for the syntax :

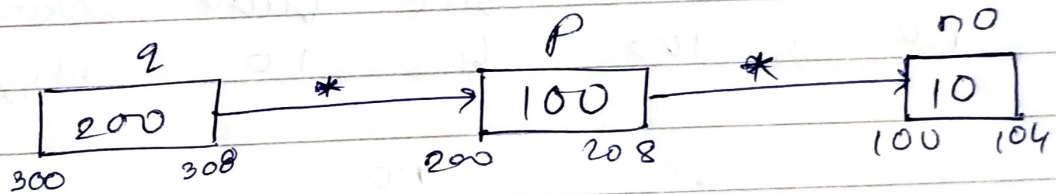
1] int no = 10; // 100

int *p = &no; // 200

int **q = &p; // 300

Name	Address	Size	Value	Datatype	Another Name
no	100	4	10	integer	*p, **q
*p	200	8	10	integer pointer	&no.
*q	300	8	10		&p

5]



6] Draw symbol table & diagrammatic representation of below syntax

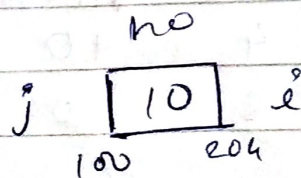
→ int no = 10; // Consider address = 100

int &i = no;

int &j = no;

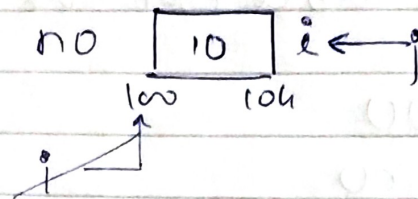
Name	Address	Size	Value	Datatype	Another name
no	100	4	10	integer	i, j

Diagrammatic Representation :



7] `int no = 10; // Address of no = 100`
`int &i = no; // Ref to no`
`int &j = i; // R-f to no via i`

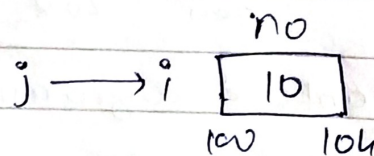
Name	Address	Size	Value	Datatype	Another name
no	100	4b	10	integer	i, j



8] Draw symbol-table & diagrammatic representation of below syntax

`int no = 10; // 100`
`int &i = no;`
`int &j = i;`

Name	Address	Size	Value	Datatype	Another name
no	100	4	10	integer	i, j



9] Draw symbol-table & diagrammatic representation

`int no = 10; // 100`
`int *i = &no; // 100`
`int *(&j) = i;`

Name	Address	Size	Value	Datatype	Another name
no	100	4	10	integer	i, *(&j)
i	200	8	10	integer	&no



10] What is mean by call by value & call by reference?
 → Call by value:

In call by value method, function arguments are passed by copying the value of actual parameter, ensuring the original values remain unchanged.

The value is copied to the formal parameter.

eg. #include <iostream>

using namespace std;

void increment (int num)

{ num ++;

cout << num << endl; }

int main ()

{ int number = 5;

increment (number); // passing number by value

cout << number << endl;

return 0;

}

6] Call by reference:

→ The memory address (ref) of actual parameter is passed to function, allowing direct access & modification of the original values.

- The actual & formal parameters point to the same memory address.

- Any changes made to parameters within function are directly reflected in original values outside the function.